

MISAKA: 検証済み LLM トークン重み付き BFT

Bonded Validators による Useful Compute Consensus

watanabe misaka

info@misakachain.com

2026 年 8 月 7 日

要旨

本稿は、Validator が Bond した資本量ではなく、直近に提供した検証済み **LLM** トークン量を投票重みとして用いる Byzantine Fault Tolerant consensus を提案する。LLM job はモデル、runtime、数値形式、入力 commitment、乱数 seed を固定した仕様として割り当てられ、実行結果が独立検証された場合にのみ Verified LLM Token (VLT) を生成する。Epoch E で確定した VLT は Epoch $E + 1$ 以降の重みに反映され、現在の fork 上で得た計算が同じ fork の投票力を直ちに増やす循環を防ぐ。Validator は MISAKA token を Bond するが、Bond 自体は投票力を生成せず、VLT 重みに対する slash 可能な担保として機能する。各 Block は、Epoch 内の総有効 VLT 重みの $\frac{2}{3}$ を超える Precommit によって確定する。Byzantine 重みが $\frac{1}{3}$ 未満であり、署名、hash、計算検証が安全である限り、競合 Block は同時に確定しない。

1. はじめに

分散型台帳は、誰が Block を提案できるかだけでなく、複数の提案のうち何を正史として確定するかを決めなければならない。Proof of Work は hash 計算、Proof of Stake は slash 可能な資本を Sybil resistance と投票力の基礎にする [1, 4]。本方式は、投票力の源泉を最近ネットワークへ提供された検証済み **LLM** 計算へ置き換える。

単なる生成 token 数の自己申告は利用できない。モデル規模、active parameters、context 長、精度、runtime が異なれば、同じ 1 token でも必要計算量が異なる。また、現在の Block 上で得た計算 credit を同じ Block の投票へ使用すると、fork と投票力が相互に増幅する。本方式は、(i) consensus で固定された計算仕様、(ii) 独立した計算証明、(iii) Epoch 遅延、(iv) 有効期限付き credit、(v) Bond と slash を組み合わせる。

専用の LLM miner、恒久的な Verifier A/B、単一 Beacon は存在しない。Bond 済み参加者は job ごとに Executor または Verifier となり、次 Epoch では確定済み VLT に応じた Validator weight を持つ。Block consensus と LLM job execution は分離されるため、全 Validator が各 Block ごとに同一 LLM を再実行する必要はない。

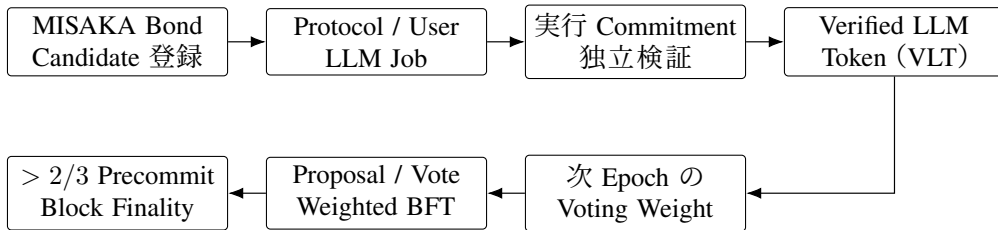


Figure 1: Verified Compute から次 Epoch の BFT 投票力までの流れ。

2. システムモデル

ネットワークは最終的に message 遅延の上限が成立する partial synchrony を仮定する [3, 5, 6]。Hash 関数は collision resistant、署名は unforgeable であるとする。各 Validator i は consensus key、Bond 残高 B_i 、計算 credit を持つ。

時間は Epoch $E = 0, 1, 2, \dots$ に分割され、各 Epoch は複数の Block height と round からなる。Validator set と weight は Epoch 中に固定され、Epoch 境界の確定 Block に次 Epoch の set commitment が記録される。Candidate は所定の minimum Bond を lock した後、投票力がゼロでも LLM job を実行できる。Epoch E で確定した計算は、最短でも Epoch $E + 1$ の投票にのみ使用される。Genesis では事前定義した初期 set または計算 bootstrap 期間を用いる。

3. Verified LLM Token

3.1 Job specification

Consensus weight を生成できる job は、実行前に次の JobSpec を固定する。

$$S_j = (h_M, h_R, q, p_j, s_j, L_j, v_j), \quad (1)$$

ここで、 h_M は model weights、 h_R は runtime、 q は precision または quantization profile、 p_j は input commitment、 s_j は sampling seed、 L_j は token 上限、 v_j は verification scheme を表す。Tokenizer、sampling rule、終了条件も S_j に含まれる。Consensus-eligible な profile は、同じ仕様に対して protocol が要求する検証関係を再現できなければならない。

Executor は output token 列またはその commitment、token count、実行 trace commitment を含む Receipt を作る。

$$R_j = H(S_j \parallel h_{y_j} \parallel t_j^{\text{in}} \parallel t_j^{\text{out}} \parallel z_j). \quad (2)$$

Receipt が登録済み verification relation を満たし、challenge 期間を通過したとき、Compute Certificate C_j が成立する。

$$\text{Verify}(S_j, R_j, C_j) = 1. \quad (3)$$

3.2 Token normalization

Raw token 数をそのまま比較せず、network が定める reference token-equivalent へ変換する。Job j が生成する VLT を

$$x_j = \begin{cases} \rho(S_j)(a t_j^{\text{in}} + b t_j^{\text{out}}), & \text{Verify}(S_j, R_j, C_j) = 1, \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

とする。 $\rho(S_j)$ は model の active parameters、context、precision、sparsity、canonical benchmark cost を反映する consensus parameter であり、Executor は変更できない。 a と b は prefill と decode の相対 cost である。VLT は Validator key に結び付いた非譲渡 credit であり、token として送金または委任できない。

4. Epoch weight

Epoch e で Validator i に確定した VLT の合計を $X_i(e)$ とする。Epoch E で用いる recent compute score は

$$C_i(E) = \sum_{\tau=1}^K d_\tau X_i(E - \tau), \quad 1 = d_1 \geq d_2 \geq \dots \geq d_K > 0 \quad (5)$$

である。 K は credit window、 d_τ は減衰係数である。したがって、古い計算は徐々に投票力を失い、停止した設備が永久に支配力を保持しない。

Bond は投票力の加算項ではない。ただし、大きな投票力が小さな担保だけで得られると slash の経済的意味が失われるため、有効 weight を

$$W_i(E) = \min\{C_i(E), \lambda B_i(E)\} \quad (6)$$

とする。 λ は 1 Bond 単位が担保できる最大 VLT weight である。Bond だけを増やしても $C_i(E) = 0$ なら $W_i(E) = 0$ であり、投票力は生じない。一方、VLT を大量に獲得しても十分な slashable Bond がなければ、その全量を投票力へ変換できない。

総 weight と quorum threshold は

$$W(E) = \sum_{i \in \mathcal{V}_E} W_i(E), \quad Q(E) = \left\lceil \frac{2W(E)}{3} \right\rceil + 1 \quad (7)$$

とする。実装では weight を整数化し、単なる「ちょうど 3 分の 2」ではなく $Q(E)$ 以上を要求する。新しい Validator set は $W(E)$ が minimum network compute $W_{\min}$ 以上の場合にのみ有効化される。下回る場合、set transition を延期し、recovery rule へ移行する。

5. Weighted BFT consensus

各 height h では round $r = 0, 1, 2, \dots$ を進める。Leader は前 Epoch の確定状態から得た consensus randomness を用い、 $W_i(E)/W(E)$ に比例する確率または同等の weighted schedule で選ばれる。単一 Validator が randomness を決定してはならない。

各 round は次の 4 段階からなる。

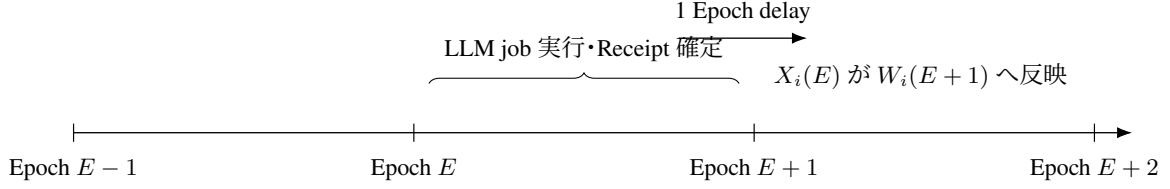


Figure 2: 計算 credit と投票 weight の時間分離。

1. **Proposal:** Leader は parent finality proof、transaction root、state root、validator-set hash を含む Block B を提案する。
2. **Prevote:** Validator は有効な Block または nil へ、同一 height・round につき一度だけ Prevote する。
3. **Lock and Precommit:** Block B への Prevote weight が $Q(E)$ 以上になると Prevote Certificate が成立する。Validator は B を lock し、 B へ Precommit する。より高い round の有効 Certificate がない限り、競合 Block へ lock を移さない。
4. **Finality:** B への Precommit weight が $Q(E)$ 以上になれば、 B は確定する。timeout 時は round を進める。

署名集合 A の weight を $w(A) = \sum_{i \in A} W_i(E)$ と書くと、Finality Certificate は

$$QC(B, h, r) = \{\sigma_i(B, h, r)\}_{i \in A}, \quad w(A) \geq Q(E) \quad (8)$$

である。人数の 3 分の 2 ではなく、確定済み VLT に由来する **weight** の 3 分の 2 超が必要となる。Weight は Epoch 中に変化しないため、同じ height の参加者は同一の quorum denominator を使用する。

6. Compute certification

恒久的な Executor 役と Verifier 役は置かない。Job ごとに Bond 済み Candidate から Executor と Verifier を分離し、Executor の commit 後に確定 chain 由来の randomness で Verifier を選ぶ。これにより、Executor が事前に協力者だけを選ぶことを防ぐ。

Verification scheme v_j は、canonical full replay、commitment に対する randomized trace challenge、または succinct proof を利用できる。ただし、scheme の受理条件は consensus code で決定的でなければならない。Verifier は自分の job を検証できず、互いに矛盾する Certificate へ署名した場合は slash 対象となる。Executor は Job fee と VLT を受け取り、Verifier は audit fee を受け取るが、同じ job の複製検証を Useful Compute として二重計上しない。

LLM 推論が hardware 間で非決定的になる profile は、そのまま consensus credit へ使用しない。Token IDs、sampling seed、runtime、precision を固定して再現可能にするか、実行の正しさを直接証明する必要がある。一般的な非決定的 LLM service は network 上で提供できるが、検証関係を満たさない限り VLT を生成しない。

Receipt は Epoch 終了前に確定 Block へ入り、challenge window を経て初めて $X_i(E)$ へ加算される。不正証明が成立した Receipt は credit をゼロにし、Executor および不正 Certificate へ署名した Verifier を slash する。この遅延により、現在の fork で作った虚偽 credit を現在の投票へ即時使用できない。

7. Incentive and slashing

Validator の収益は、LLM job fee、verification fee、Block proposal および vote reward から構成できる。Reward の配分は投票 weight の定義と分離する。VLT は市場で移転できず、recent compute window を外れると失効するため、過去の計算 credit を恒久資産として売買することはできない。

次の行為は署名または commitment によって客観的に証明できる slash offense である。

- (a) 同一 height・round で競合する Prevote または Precommit へ署名すること。
- (b) 無効な Compute Certificate、矛盾する verification result、または偽造 Receipt へ署名すること。
- (c) Challenge に失敗した実行を正しいものとして claim すること。

有効な Evidence が確定すると、対象 Validator の active weight をゼロにし、Bond の一部または全部を slash し、一定期間 set から除外する。Unbonding delay U は、少なくとも credit window と最大 challenge 期間を超えなければならない。単なる offline は必ずしも equivocation と同じ slash 率にせず、missed reward、inactivity penalty、VLT decay によって処理できる。

8. Security

8.1 Safety

Epoch E の Byzantine weight が $W(E)/3$ 未満であると仮定する。同じ height で競合する Block B と B' が確定したなら、それぞれの Precommit set A と A' は $2W(E)/3$ を超える。したがって

$$w(A \cap A') > \frac{W(E)}{3}. \quad (9)$$

Byzantine weight が 3 分の 1 未満なら、intersection には少なくとも 1 つの honest weight が含まれる。Honest Validator は one-vote rule と lock rule に従うため、競合する 2 つの Finality Certificate へ有効に参加できない。これは仮定に反する。よって、暗号と計算検証が安全である限り、競合 Block は同時に確定しない [4, 5, 6]。

8.2 Liveness

Network が eventually synchronous となり、Byzantine および offline weight の合計が 3 分の 1 未満なら、timeout は最終的に message delay を上回る。Honest Leader の round では有効 Proposal が 3 分の 2 超の Prevote と Precommit を集め、chain は進行する。攻撃者が 3 分の 1 以上の weight を支配すると、署名を拒否するだけで liveness を停止できる。3 分の 2 超を支配すると、攻撃者だけで Certificate を作れる。

攻撃者 weight	結果
$< 1/3$	Partial synchrony と honest lock を仮定し、Safety と Liveness を維持する。
$\geq 1/3$	Vote 拒否により Liveness を停止できる。Safety 保証の前提も失われる。
$> 2/3$	攻撃者だけで Prevote / Precommit Certificate を形成できる。

Table 1: Weighted BFT における主要 threshold。51% は特別なしきい値ではない。

8.3 Compute fraud and feedback

自己申告 token は weight にならず、valid Certificate を持つ VLT だけが計上される。Job assignment を commit 後に決め、Receipt を challenge 可能にすることで、事前共謀と架空計算を抑える。さらに、Equation (5) の Epoch delay によって、ある fork 上で生成した credit は同じ fork の即時支配に利用できない。Equation (6) は、獲得した compute weight を slash 可能な担保へ結び付ける。

8.4 Long-range and concentration

この方式は資本保有量を投票力にしないが、署名型 BFT である以上、古い key を用いた long-range history の問題を無条件には消さない。Consensus key を Epoch ごとに rotation し、使用済み key を削除し、Unbonding 期間内の最新 finalized checkpoint から同期する。新規 node は少なくとも Unbonding horizon より新しい checkpoint を取得する必要がある。

また、linear VLT weight は compute 集中をそのまま consensus 集中へ変換する。単一 operator が総 VLT の 3 分の 1 を得れば停止能力を持ち、3 分の 2 超を得れば単独確定能力を持つ。Identity 単位の上限は Sybil 分割で回避できるため、それだけを安全策とみなしてはならない。Model table、job demand、hardware 供給、Verifier 独立性は consensus security parameter である。

9. Block and execution layer

本方式は UTXO、EVM、または他の state transition と独立である。Block header は少なくとも parent、transaction root、state root、VLT receipt root、Validator-set hash、Finality Certificate を commit する。

$$H_h = H(H_{h-1} \parallel r_{tx} \parallel r_{state} \parallel r_{VLT} \parallel h_V \parallel QC_{h-1}). \quad (10)$$

DAG を transaction または proposal の伝播層として利用することもできるが、canonical history は Weighted BFT で確定した ordering によって決まる。DAG 内の到着順や単一 Beacon は Finality を上書きしない。

10. 結論

MISAKA Verified LLM Token-Weighted BFT では、Validator は MISAKA token を Bond して参加し、protocol が検証した LLM token-equivalent を獲得する。Recent VLT は次 Epoch の weight となり、その 3 分の 2 超の

Precommit によって Block が確定する。Bond は投票力を購入する資本ではなく、不正時に失われる担保であり、VLT weight を経済的に裏付ける。これにより、PoS の「Money $\rightarrow$ Power」を「Verified Useful Compute $\rightarrow$ Power」へ置き換えつつ、BFT の quorum intersection、finality、equivocation slashing を利用できる。

本稿は consensus の最小理論を定義する。Production deployment には、VLT normalization table、deterministic runtime、verification scheme、randomness、parameter governance、network recovery を個別に仕様化し、formal verification と adversarial test を行う必要がある。

参考文献

- [1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
- [2] L. Lamport, R. Shostak, and M. Pease, “The Byzantine Generals Problem,” *ACM Transactions on Programming Languages and Systems*, 1982.
- [3] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the Presence of Partial Synchrony,” *Journal of the ACM*, 1988.
- [4] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance,” *OSDI*, 1999.
- [5] E. Buchman, J. Kwon, and Z. Milosevic, “The Latest Gossip on BFT Consensus,” 2018.
- [6] M. Yin, D. Malkhi, M. K. Reiter, G. Golan Gueta, and I. Abraham, “HotStuff: BFT Consensus with Linearity and Responsiveness,” *PODC*, 2019.